

Ex3

Aim

To implement and compare three different methods of interacting with a computer (CLI, GUI, and VUI) using a Python script that renames files.

1. Command Line Interface (CLI)

Aim: To rename a file by typing specific text commands into a terminal.

Procedure:

- **Step 1:** Import os and sys modules to handle file operations and command-line arguments.
- **Step 2:** Write a function that uses os.rename() to change the filename.
- **Step 3:** Use sys.argv to capture the old and new filenames directly from the user's typed command.
- **Step 4:** Run the script by typing: python rename_file.py old.txt new.txt.

Code:

```
import os
import sys

def rename_file(old_name,new_name):
    try:
        os.rename(old_name,new_name)
        print(f"File renamed from {old_name} to {new_name}")
    except FileNotFoundError:
        print(f"Error:{old_name} not found.")
    except Exception as e:
        print(f"An error occured:{e}")

if __name__=="__main__":
    if len(sys.argv)!=3:
        print("Usage: python renmae_file_cli.py<old_filename>
<new_filename>")
    else:
        rename_file (sys.argv[1], sys.argv[2])
```

2. Graphical User Interface (GUI)

Aim: To rename a file using visual elements like text boxes and buttons.

Procedure:

- **Step 1:** Import tkinter to create a windowed application.
- **Step 2:** Create a window (root) with labels and two input fields (Entry widgets).
- **Step 3:** Add a "Rename File" button that triggers the rename function when clicked.
- **Step 4:** The user types the names into the boxes and clicks the button.

Code:

```
import tkinter as tk
from tkinter import messagebox
import os

def rename_file():
    old_name = old_filename_entry.get()
    new_name = new_filename_entry.get()

    try:
        os.rename(old_name, new_name)
        messagebox.showinfo("Success", f"File renamed from {old_name} to {new_name}")
    except FileNotFoundError:
        messagebox.showerror("Error", f"File {old_name} not found.")
    except Exception as e:
        messagebox.showerror("Error", f"An error occurred: {e}")

root = tk.Tk()
root.title("File Renamer")

tk.Label(root, text="Old Filename:").grid(row=0, column=0)
tk.Label(root, text="New Filename:").grid(row=1, column=0)

old_filename_entry = tk.Entry(root)
old_filename_entry.grid(row=0, column=1)

new_filename_entry = tk.Entry(root)
new_filename_entry.grid(row=1, column=1)

rename_button = tk.Button(root, text="Rename File", command=rename_file)
rename_button.grid(row=2, columnspan=2)
root.mainloop()
```

3. Voice User Interface (VUI)

Aim: To rename a file using spoken verbal commands.

Procedure:

- **Step 1:** Import speech_recognition to translate voice into text.
- **Step 2:** Use the computer's microphone to listen for the user's speech.
- **Step 3:** Convert the audio into strings (text) for the "Old Filename" and "New Filename."
- **Step 4:** Clean the text (replace spaces with underscores) and perform the rename operation.

Code:

```
import speech_recognition as sr
import os

def rename_file_from_voice_command(old_name, new_name):
    """Renames a file using the provided old and new names."""
    try:
        # Adding extension automatically as per your logic
        old_file = old_name + ".txt"
        new_file = new_name + ".txt"

        if not os.path.exists(old_file):
            print(f"❌ Error: '{old_file}' not found.")
            return

        os.rename(old_file, new_file)
        print(f"✅ File successfully renamed from '{old_file}' to '{new_file}'")

    except Exception as e:
        print(f"❌ Error: {e}")

def listen_for_filename(prompt):
    """Listens for a single filename input via voice command."""
    recognizer = sr.Recognizer()
    mic = sr.Microphone()

    with mic as source:
        # Adjust for background noise
        recognizer.adjust_for_ambient_noise(source, duration=2)
        print(f"🎧 {prompt}")

    try:
```

```

        audio = recognizer.listen(source, timeout=10, phrase_time_limit=5)
        # Convert speech to text using Google's engine
        command = recognizer.recognize_google(audio, language="en-US")
        print(f"🗣️ You said: {command}")

        # Clean the text: remove spaces and lowercase it
        return command.strip().replace(" ", "_").lower()

    except sr.UnknownValueError:
        print("❌ Could not understand. Please try again.")
        return None
    except sr.WaitTimeoutError:
        print("⌚ Timeout: No speech detected.")
        return None

if __name__ == "__main__":
    print("🎧 Welcome to the Voice-Controlled File Renamer!")

    # Get the original filename
    old_name = None
    while old_name is None:
        old_name = listen_for_filename("Say the name of the file to rename (without .txt)")

    # Get the new filename
    new_name = None
    while new_name is None:
        new_name = listen_for_filename("Say the NEW name for the file (without .txt)")

    # Execute the rename
    rename_file_from_voice_command(old_name, new_name)

```